ReceiptOS Verify x XRPL

A Minimal Evidence-Anchoring Adapter Using Existing XRPL Primitives

Technical discussion note for Florian Alonso / XRPL Commons

1. Summary

XRPL can establish that a commitment was submitted under an account's authorization and included in a validated ledger. ReceiptOS Verify independently determines what the referenced evidence establishes for the caller-selected claim. **Ledger inclusion != semantic correctness. Transaction effect/result != anchor inclusion. Valid anchor for claim A != proof of claim B.** [1,2]

The public v0.2.0 baseline checks bounded claim/evidence compatibility, integrity, authenticity, recomputation, sufficiency and requested guarantees, preserving native results. Its ledger boundary is interface-only; no live XRPL adapter exists. [9]

2. Why this boundary matters

Public Semantic ABI asks what one system's evidence is allowed to mean when consumed by another. ReceiptOS operationalizes that boundary with separate claim, subject/artifact, authority, scope, policy/version, time, assumptions, dependencies, evidence basis and replay checks. These are distinct projects; a digest does not replace their semantic relation. Current scope comparison is conservative equality, not arbitrary entailment. [9,10]

3. Minimal xrpl-receipt-anchor-v0

CURRENT XRPL PRIMITIVES: a standalone `AccountSet` with no account-setting options, authorized normally, carrying a Memo. V0 selects ordinary Sequence (no Ticket): it consumes a fee/sequence without changing account settings. [3,4]

PROPOSED RECEIPTOS CONVENTION: one anchor Memo selected by exact decoded `MemoType = urn:receiptos:xrpl-receipt-anchor-v0`; `MemoFormat = application/octet-stream`; `MemoData = R[32] || C[32]`. All three are hex-encoded for XRPL JSON. Type fixes layout/version; R is the supported SHA-256 receipt root, not a hash of arbitrary exported JSON. C commits to the full declared claim surface, verification/root profiles and network/checkpoint. Subject stays in that preimage. No raw evidence or verdict goes on-ledger. XRPL does not require this URN.

Crystal R excludes top-level anchor and is not whole-envelope identity; authenticated statement and payload/manifest closure must also pass.

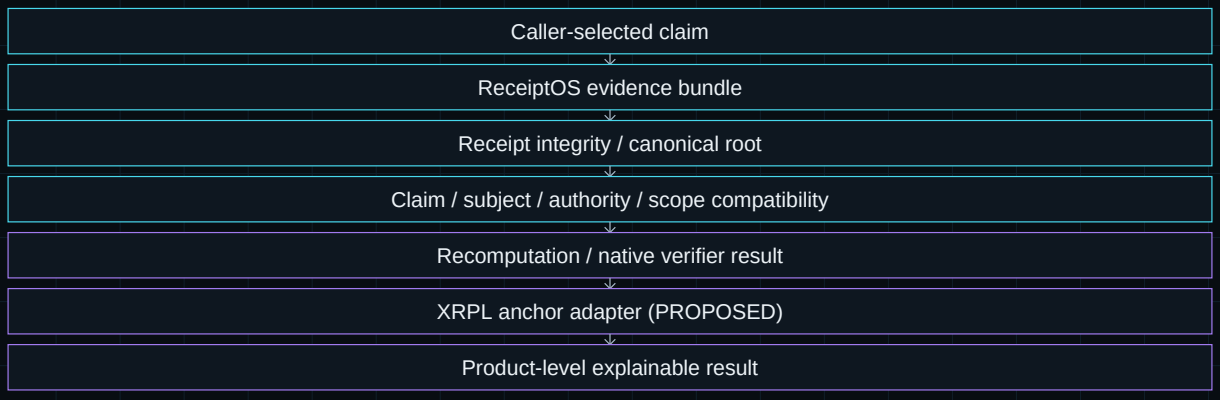
Define `C = SHA256(domain || UTF8(JCS(preimage)))`, domain ASCII `receiptos.xrpl-receipt-anchor-v0.claim` followed by one NUL. JCS is RFC 8785: no duplicate keys, nonfinite numbers or invalid Unicode; no Unicode normalization; array order matters. Retain v0.2 safe-integer/schema bounds. JCS is used only for the portable XRPL claim commitment so independent non-ReceiptOS implementations can derive the same bytes from a published standard; it does not replace ReceiptOS' internal canonicalization profile. [8]

Decoded data is 64 bytes; type/format are 36/24 bytes. All binary-serialized Memos together must fit XRPL's 1 KB limit, including overhead. Zero matching Types -> NOT_ESTABLISHED; more than one -> MALFORMED. Preserve/ignore unrelated Memos; never select first/last. Reject malformed matching Memo bytes/length. Unsupported requested profiles or delegated/sponsored/inner-Batch carriers remain unsupported. [3]

Compare Memo bytes to receipt-derived R/C; bind actual transaction hash, accepted Account and validated ledger hash/index to the caller-trusted network/checkpoint history. Exact accepted anchor in a trusted validated transaction -> ANCHOR_MATCH for either `tesSUCCESS` or `tec` result. Preserve `meta.TransactionResult` separately. Any policy requiring `tesSUCCESS` is a separate product obligation; it cannot erase inclusion. A saved RPC flag is not proof. Transaction ID uses XRPL signed-binary hashing, not SHA-256 of RPC JSON. [1,2,5]

`NetworkID` is omitted for IDs <=1024 (Mainnet 0); required >=1025. Its absence is not Mainnet identity. Check the selected chain independently; IDs alone do not prevent every cross-chain replay. Master/regular-key/multisign account authorization does not establish the receipt issuer's business authority. [3,6]

4. Verification flow



Proposed anchor-native states: `ANCHOR_MATCH`, `ANCHOR_MISMATCH`, `ANCHOR_NOT_ESTABLISHED`, `ANCHOR_MALFORMED`, `ANCHOR_UNSUPPORTED`. Preserve XRPL metadata separately. MATCH means the declared anchor relation holds under stated ledger trust; it never maps directly to global VERIFIED. Missing history/untrusted observation is NOT_ESTABLISHED, not proof of non-inclusion. [2]

5. Two concrete examples

Both examples assume independently established receipt integrity and a trusted validated transaction anchoring receipt R and declaration A. They are synthetic contract examples, not ledger observations.

A - matching claim: receipt covers commit A; caller asks for A. Anchor = ANCHOR_MATCH; claim/subject = COMPATIBLE. Remaining layers continue; no global VERIFIED is inferred from the anchor.

B - wrong claim: same receipt and same valid anchor; caller asks for B. Anchor = ANCHOR_MATCH; integrity = VERIFIED; claim/subject = CONTRADICTED; overall = CONTRADICTED. The ledger anchor remains valid; transferring its support to B is invalid. This does not prove that B executed or failed.

6. DID / Credentials extension (optional)

DID can supply account-controlled document/verification-method references; XRPL does not validate that document's meaning. Credentials bind issuer account, subject account and issuer-defined CredentialType (1-64 bytes). A future policy must check subject acceptance, expiry and deletion/revocation at a selected ledger, plus accepted issuer/type. Neither establishes execution correctness; neither is mandatory for v0. Network amendment enablement must be checked before use. [7]

7. When would a new XRPL primitive be justified?

Consider a protocol primitive only after reproducing a needed relation that existing primitives cannot represent or verify without ambiguity, unsafe overloading or unacceptable off-ledger convention dependence. **POSSIBLE GAP - NOT YET ESTABLISHED:** typed claim/evidence commitments; explicit authority/subject relations; profile discoverability; stronger canonical semantics than Memo conventions. Implement and test the adapter first; derive any protocol discussion from a concrete failure. Existing primitives appear sufficient for the first bounded anchor.

8. Current limitations

This is an architecture proposal: no live transaction, codec conformance, portable consensus proof or deployment is demonstrated. Archive/validator trust and root-profile mapping need implementation tests. Anchoring proves no external execution, provider/hardware authenticity, evidence availability or arbitrary semantic entailment. Ledger inclusion time does not establish execution time.

ReceiptOS Verify is an independent experimental project by Pavlo Tvardovskyi. It uses or interoperates with public Trustless verification primitives where applicable. This adapter is not an official Trustless-AI or XRPL Commons integration, nor a protocol commitment by either organization.

9. References

1 Finality | 2 tx | 3 Common fields | 4 AccountSet | 5 Hashes | 6 Keys | 7 DID | 7 Credentials | 8 JCS | 9 v0.2.0 | 10 Semantic ABI
Accessed 2026-10-12 UTC. Exact pins, official URLs and claim mapping: docs/xrpl/REFERENCES.md.